

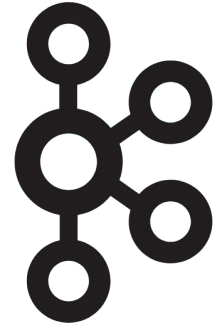
“ஒருமைக்கண் தான்கற்ற கல்வி ஒருவற்கு
எழுமையும் ஏமாப் புடைத்து. ”

**The learning acquired in one birth Protects a man in the next
seven.**

1
0
1
0
1
0
1
0
1
0
1

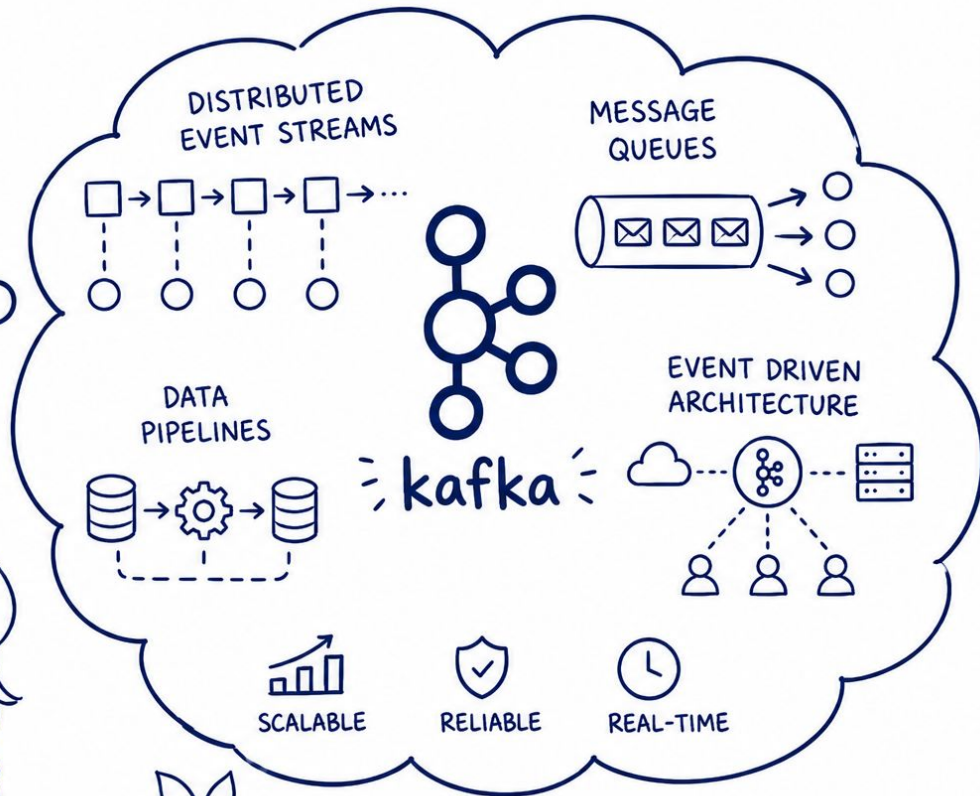
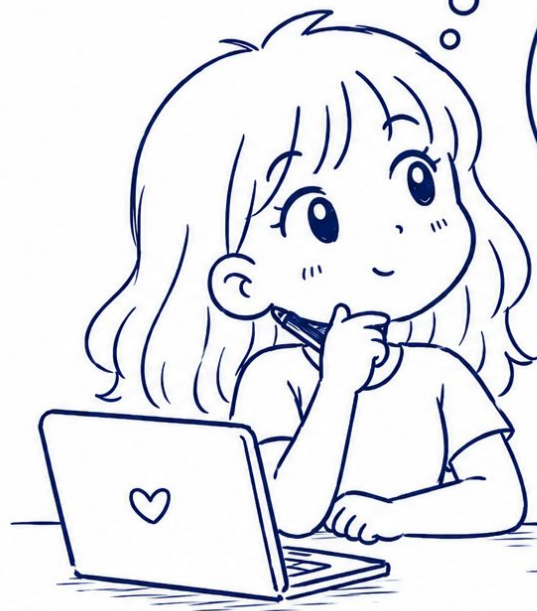
Building Event-Driven Systems with Apache Kafka

YAGAPRIYAN G



- **Why Event-Driven Architecture?**
- **Kafka Fundamentals**
- **Kafka Core Components**
- **Real-world Use Cases**
- **Java & Python Demo**

?



TRADITIONAL SYSTEMS HAVE PROBLEMS

Challenges:

- ✗ Tight coupling.
- ✗ Slow communication between applications.
- ✗ Difficult to scale.
- ✗ Failures propagate easily.
- ✗ Higher Response Time.
- ✗ Poor Scalability etc;

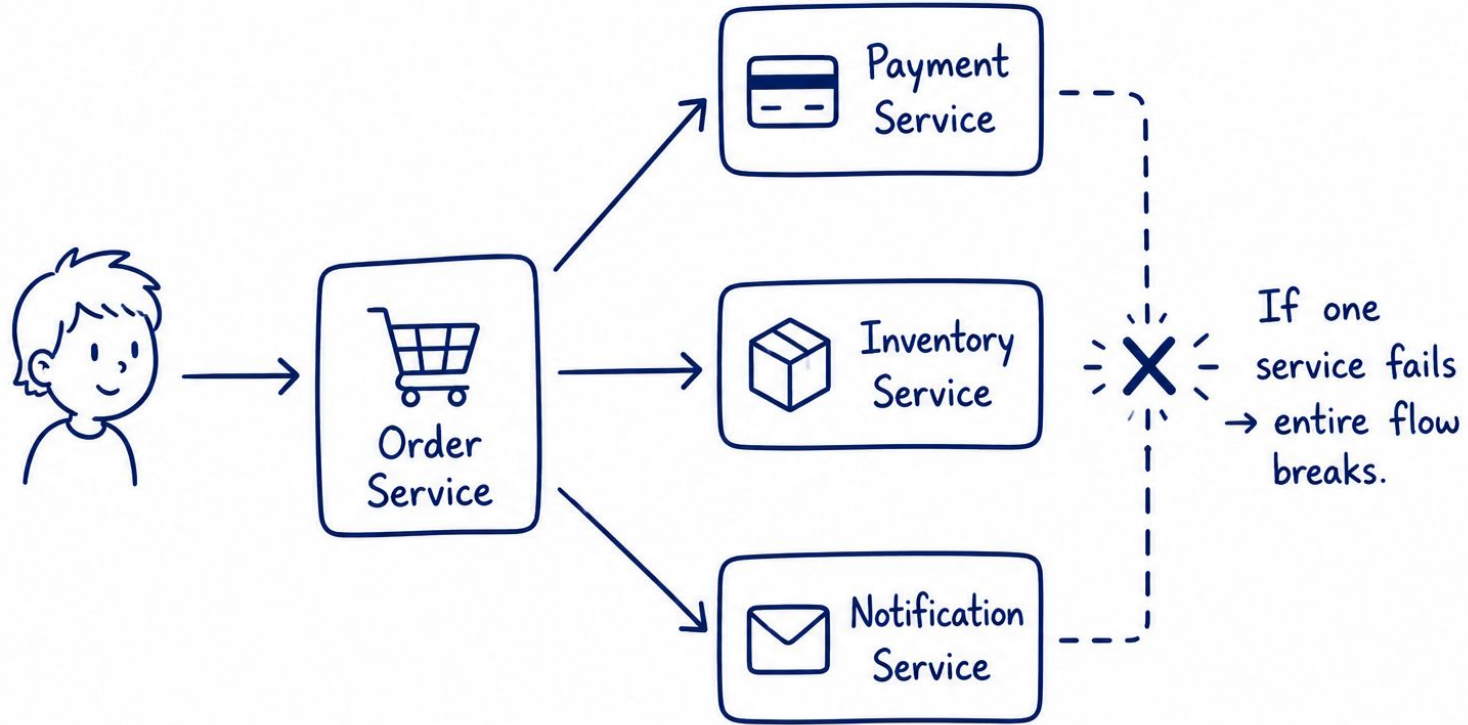
REST API

GRAPHQL

GRPC

SOAP API

Example: Synchronous (Direct Calls)



EVENT-DRIVEN ARCHITECTURE (EDA)

Applications communicate by **publishing** and **consuming events** instead of directly calling each other.

Real Life Events



OTP Sent → SMS Service.



Order Placed → Email + Inventory + Billing.



Money Credited → Notification + Analytics.



Cab Booked → Driver Matching + Tracking.

One event can trigger many actions

WHY APACHE KAFKA ?

Apache Kafka

Apache Kafka is a **distributed event streaming platform**.

Kafka provides:

- ✓ High Throughput.
- ✓ Fault Tolerance.
- ✓ Scalability.
- ✓ Real-time Processing.
- ✓ Durability.



Uber



LinkedIn

KAFKA ECOSYSTEM

PRODUCER



BROKER

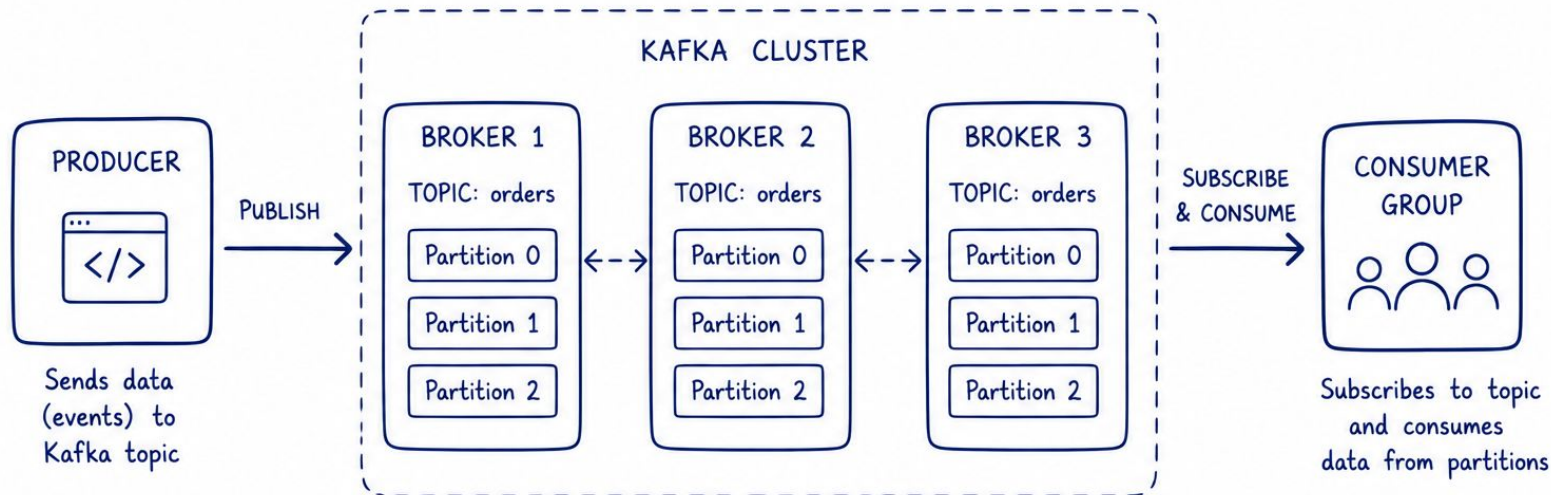


TOPIC

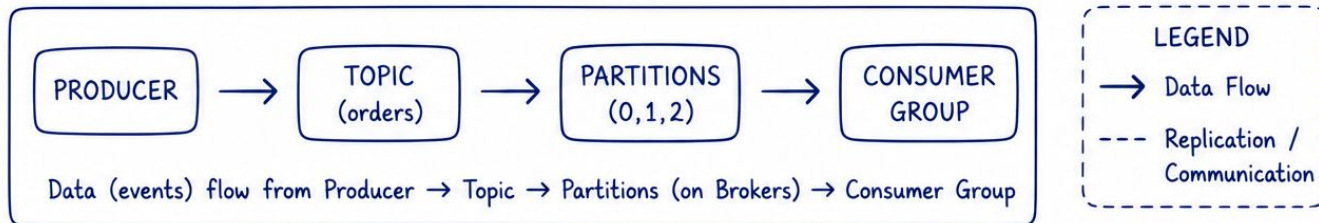


CONSUMER / CONSUMER GROUP

Kafka Core Components – Data Flow



Topic is split into partitions
and distributed across brokers



PRODUCER - Publishes events/messages to Kafka topics.

BROKER - Kafka server that stores events and serves producer/consumer requests.

TOPIC - A logical stream of related events.

PARTITION - An ordered storage unit of a topic that enables parallel processing.

CONSUMER - Reads events from one or more topic partitions.

CONSUMER GROUP - Multiple consumers working together to process messages in parallel.

FACTS:

- Producers and consumers are independent, making Kafka **loosely coupled**.
- A topic can have multiple partitions for scalability.
- Within a consumer group, **one partition is assigned to only one consumer at a time**.
- A **consumer can consume from multiple partitions**, but a partition cannot be shared by multiple consumers in the **same consumer group**.
- Multiple consumer groups can independently consume the same topic. This is what allows different applications (e.g., Analytics, Notifications, Billing) to process the same events without affecting each other.



DEMO...



QUESTIONS ?

THANK YOU



Let's Connect 